

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- Analyze the CSS properties responsible for layout overlapping issues.
- Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

*I **Ch.Sathish** / 192311032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Ch.Sathish

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

Experiment 1: Responsive Layout Design

(a) Analyze the possible reasons for lack of responsiveness

Common reasons include:

- Fixed-width layouts using pixels (width: 1000px) instead of flexible units (% , vw, rem).
- Missing viewport meta tag.
- Using floats instead of modern layout techniques like Flexbox or Grid.
- Images without responsive sizing (max-width: 100%).
- No media queries for different screen sizes.
- Improper spacing and alignment using absolute positioning.

(b) Design a responsive webpage layout using CSS Flexbox

HTML

```
<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Responsive Layout</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<header>

  <h1>Retail Website</h1>

</header>

<nav>

  <a href="#">Home</a>
```

Products

About

Contact

</nav>

<div class="container">

<aside>Sidebar</aside>

<main>

<h2>Welcome</h2>

<p>This is the main content section.</p>

</main>

</div>

<footer>

© 2026 Retail Website

</footer>

</body>

</html>

CSS

```
*{
  margin:0;
  padding:0;
  box-sizing:border-box;
}
body{
  font-family:Arial,sans-serif;
}
header,footer{
  background:#333;
  color:white;
  text-align:center;
  padding:20px;
}
nav{
  display:flex;
  justify-content:center;
  background:#555;
```

```

}
nav a{
  color:white;
  text-decoration:none;
  padding:15px;
}

.container{
  display:flex;
  gap:20px;
  padding:20px;
}
aside{
  flex:1;
  background:#f2f2f2;
  padding:20px;
}
main{
  flex:3;
  background:#e8f4ff;
  padding:20px;
}

```

(c) Media Queries

```

/* Tablet */
@media(max-width:768px){
  .container{
    flex-direction:column;
  }

  nav{
    flex-wrap:wrap;
  }
}

/* Mobile */
@media(max-width:480px){

  nav{
    flex-direction:column;
    text-align:center;
  }

  header,footer{
    font-size:18px;
  }

  main,aside{
    padding:15px;
  }
}

```

```
}  
}
```

Layout Adaptation

- **Desktop:** Navigation in one row, sidebar beside content.
- **Tablet:** Sidebar moves below content.
- **Mobile:** Navigation becomes vertical and all sections stack.

(d) Spacing, Alignment and Distribution

```
.container{  
  display:flex;  
  justify-content:space-between;  
  align-items:flex-start;  
  gap:20px;  
}
```

```
nav{  
  display:flex;  
  justify-content:space-evenly;  
}
```

```
main,aside{  
  border-radius:8px;  
}
```

(e) Evaluation

Advantages

- Responsive on all devices.
- Better readability.
- Consistent spacing.
- Modern Flexbox layout.
- Easier maintenance.

Further Improvements

- Use CSS Grid for complex layouts.
- Add responsive images.
- Include dark mode.
- Optimize loading speed.
- Improve accessibility with semantic HTML and ARIA labels.

Experiment 2: Form Validation using JavaScript

(a) Analyze the Issues

Problems in existing validation:

- Invalid email addresses accepted.
- Phone numbers with alphabets accepted.
- Empty fields submitted.
- No real-time feedback.
- Poor user experience.

(b) Design Input Fields

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
</head>
<body>

<form>

<label>Email</label><br>
<input type="email" id="email" placeholder="Enter Email">
<span id="emailError"></span>

<br><br>

<label>Phone Number</label><br>
<input type="text" id="phone" maxlength="10" placeholder="Enter Phone">
<span id="phoneError"></span>

<br><br>

<input type="submit">

</form>

<script src="script.js"></script>

</body>
</html>
```

(c) JavaScript Validation

```
const email = document.getElementById("email");

const phone = document.getElementById("phone");

email.addEventListener("input", validateEmail);
```

```

phone.addEventListener("input", validatePhone);

function validateEmail(){

let pattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

if(pattern.test(email.value)){

document.getElementById("emailError").innerHTML="Valid Email";

document.getElementById("emailError").style.color="green";

}

else{

document.getElementById("emailError").innerHTML="Invalid Email";

document.getElementById("emailError").style.color="red";

}

}

function validatePhone(){

let pattern=/^[6-9]\d{9}$/;

if(pattern.test(phone.value)){

document.getElementById("phoneError").innerHTML="Valid Phone";

document.getElementById("phoneError").style.color="green";

}

else{

document.getElementById("phoneError").innerHTML="Invalid Phone";

document.getElementById("phoneError").style.color="red";

}

}

```

(d) Real-Time Error Messages

Example output:

- Email: Invalid Email
- Phone: Invalid Phone Number

The messages update immediately while typing using the input event.

(e) Evaluation

Advantages

- Prevents invalid data submission.
- Real-time feedback improves usability.
- Regular expressions provide accurate validation.
- Reduces server-side errors.

Suggested Enhancements

- Validate all required fields.
- Disable submit button until inputs are valid.
- Add password strength validation.
- Use server-side validation for security.
- Sanitize user inputs to prevent attacks.

Experiment 3: CSS Layout Debugging

(a) Analyze Overlapping Issues

Possible causes:

- Excessive use of position:absolute.
- Fixed widths causing overflow.
- Missing flex-wrap.
- Incorrect z-index.
- Overflow hidden improperly.
- Lack of media queries.

(b) CSS Box Model

The CSS Box Model consists of:

- **Content:** Actual text or image.
- **Padding:** Space inside the border.
- **Border:** Surrounds the content and padding.
- **Margin:** Space outside the border.

Example:

```
.box{  
  
    width:250px;  
  
    padding:20px;  
  
    border:2px solid black;  
  
    margin:15px;  
  
}
```

(c) Debug Using Positioning

```
.box1 {  
    position:absolute;  
    top:100px;  
    left:100px;  
}
```

```
.box2 {  
    position:absolute;  
    top:120px;  
    left:120px;  
}
```

After (Solution)

```
.container{  
    display:flex;  
    flex-wrap:wrap;  
    gap:20px;  
}
```

```
.box{  
    flex:1 1 300px;  
}
```

(d) Modified CSS

```
*{  
    margin:0;  
    padding:0;  
    box-sizing:border-box;  
}
```

```
.container{  
    display:flex;  
    flex-wrap:wrap;  
    justify-content:center;  
    gap:20px;
```

```
padding:20px;
}

.box{
  background:lightblue;
  padding:20px;
  min-width:250px;
}

img{
  max-width:100%;
  height:auto;
}
```

(e) Evaluation

Results

- No overlapping during window resizing.
- Proper alignment on all screen sizes.
- Responsive layout using Flexbox.
- Better spacing and readability.

Best Practices

- Use Flexbox or CSS Grid instead of excessive absolute positioning.
- Apply box-sizing: border-box.
- Use relative units (% , rem, vw) instead of fixed pixels.
- Add media queries for responsiveness.
- Test layouts on mobile, tablet, and desktop.
- Keep HTML semantic and CSS modular for easier maintenance.